ECE2800J Mid RC Part2

Ziqian Wang

University of Michigan - Shanghai Jiaotong University

Joint Institute

Table of Contents

Lecture 6 Procedure Abstraction

Lecture 7 Recursion

Lecture 8 Function Call Mechanism

Procedure Abstraction

Concepts:

- Abstraction is a process of emphasizing the separation of "what" and "how". It helps programmers to use a function without knowing how it is implemented.
- It only provides the details that are relevant to the user, and hide the unnecessary details.

```
// In the header file, the function is declared
int add(int a, int b);
// In the cpp file, the function is defined/implemented
int add(int a, int b) {
return a + b;
}
```

Properties

- **Local:** The implementation of an abstraction is independent of any other abstraction implementation.

```
// Here the user doesn't need to know how multiply is implemented
int square(int a) {
return multiply(a, a);
}
```

- **Substitutable:** The implementation of an abstraction can be replaced by another implementation if the interface is the same and the implementation is correct.

```
// Here the implementation can be replaced by another correct implementation
int multiply(int a, int b) {
return a * b; // can be replaced by return b*a
}
```

Type Signature

The type signature of a function is the function's name, the number of parameters, and the type of each parameter. It is used to declare the function.

```
int add(int a, int b);
```

Specification Comments

Besides the type signature, a function should also have specification comments. There're usually three types of comments:

- **REQUIRES:** Preconditions that must hold, if any.
- **MODIFIES:** Variables that are modified, if any.
- **EFFECTS:** What the procedure does given legal inputs.

```
int positiveAdd(int a, int b);
// REQUIRES: a > 0, b > 0
// MODIFIES: None
// EFFECTS: Returns the sum of a and b
```

NOTE that functions with no REQUIRES clause is complete, while functions with them are partial.

Table of Contents

Lecture 6 Procedure Abstraction

Lecture 7 Recursion

Lecture 8 Function Call Mechanism

Composition

- "stopping" casebase caseto stop the recursion
- recursive step

```
int factorial(int n)
{
    if (n == 0)
    {
        return 1; // BASE CASE, recursion must have an end
    }
    return n * factorial(n - 1); // recursive step
}
```

Exercise 1

Figure out the value of i th row and j th column (i and j start from 0)

```
            1                 n=1
          1   1               n=2
        1   2   1             n=3
      1   3   3   1           n=4
    1   4   6   4   1         n=5
  1   5   10  10  5   1       n=6
1   6  15   20  15  6   1     n=7
```

Exercise 1

- Thinking Strategy
  - Base case: i == 0 or j == 0 or i == j , the value equals to 1.
  - Recursive step: The value of (i, j) = the value of (i - 1, j) + value of (i - 1, j - 1) .

```
int pascal(int i, int j){
    if(i == 0 || j == 0 || i ==j){
        return 1;
    }return pascal(i - 1, j - 1) + pascal(i - 1, j);
}
```

Exercise 2

For an automated information service a telephone company needs the digits of phone numbers to be read digit by digit. Design an algorithm allowing to rewrite an sequence of digits into words, with a space between each word, but no space at the beginning and at the end.

Exercise 3

Given a set of characters and a positive integer k, print all possible strings of length k that can be formed from the given set.
(https://www.geeksforgeeks.org/recursion-practice-problems-solutions/)
Example: set[ ] = 'a', 'b', k = 3
Output: aaa aab aba abb baa bab bba bbb

Recursion Exercise (Optional)

21. Merge Two Sorted Lists

Solved

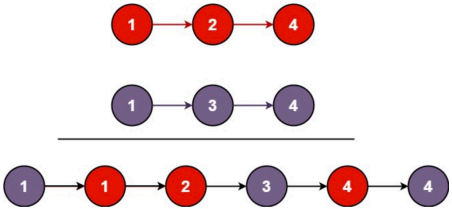
Easy | Topics | Companies

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Example 1:




```
Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
```

Recursion Exercise (Optional)

24. Swap Nodes in Pairs

Solved

Medium | Topics | Companies

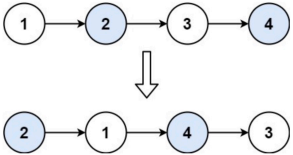
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

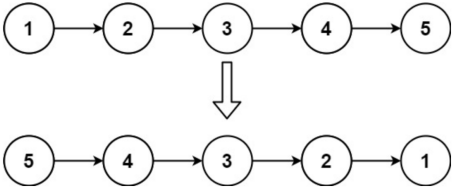
Recursion Exercise (Optional)

206. Reverse Linked List

Easy | Topics | Companies

Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]
```

Function Pointers

- **Motivation**
  - Save your time and make your code elegant.
  - min and max assigned to compareHelp
  - Treat functions as variables.
- **Grammar**

```
int (*foo)(int a, int b);
foo = max;
foo(5,3);
```

Example

Use function as function parameters:

```
void bubbleSort(int *arr, int n, bool (*cmp)(int, int)){
    for (int i = 0; i < n; i++){
        for (int j = 0; j < n - i - 1; j++){
            if (cmp(arr[j], arr[j + 1])){
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

- If you pass in bool cmp(int a, int b) { return a $<$ b; } , the array will be sorted in ascending order.
- If you pass in bool cmp(int a, int b) { return a $>$ b; } , the array will be sorted in descending order.

Question

Can we do:

```c
int max_nonconst(int a, int b) { return a > b ? a : b; }

int max_const(const int a, const int b) { return a < b ? a : b; }

int main(void) {
    int (*fp1)(const int, const int) = max_nonconst;
    const int (*fp2) (int, int) = max_nonconst;
    int (*fp3)(int, int) = max_const;
}
```

Table of Contents

Lecture 6 Procedure Abstraction

Lecture 7 Recursion

Lecture 8 Function Call Mechanism

Single Function Call Mechanism

Example:

```
int add(int a, int b){
    int result = a + b;
    return result;
}
int main(){
    int a1 = 1;
    int b1 = 2;
    int c1 = add(a1+b1, b1);
}
```

Single Function Call Mechanism

Steps to call add:

- Evaluate a1+b1 and b1 .
- Create an activation record/stack frame to hold formal parameters a and b, and local variables result.
- Copy a1+b1 and b1 to a and b respectively.
- Execute the function body.
- Replace the function call with return value result .
- Destroy the activation record.

Single Function Call Mechanism

- Stack: a set of objects which is modified as last in first out.

- When a function is called, its activation record is added to the "top" of the stack.
- When that function returns, its activation record is removed from the "top" of the stack.

JOINT INSTITUTE
交大密西根学院

Multiple Function Call Mechanism

Ordinary

Call Stacks

Example

```
int plus_one(int x) {
  return (x+1);
}

int plus_two(int x) {
  return (1 + plus_one(x));
}

int main() {
  int result = 0;

  result = plus_two(0);
  cout << result;
  return 0;
}
```

main then prints the result:

2

main:

result: 2

18

Multiple Function Call Mechanism

Pointer

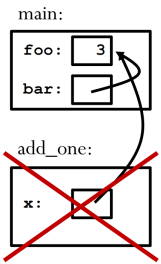
Call Stack

Example: Using Pointers

```
void add_one(int *x) {
  *x = *x + 1;
}

int main() {
  int foo = 2;
  int *bar = &foo;
  add_one(bar);
  return 0;
}
```

add_one's activation record is destroyed.



Multiple Function Call Mechanism

Recursive

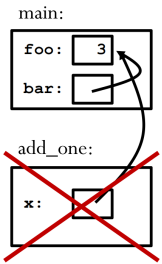
Call Stack

Example: Using Pointers

```
void add_one(int *x) {
  *x = *x + 1;
}

int main() {
  int foo = 2;
  int *bar = &foo;
  add_one(bar);
  return 0;
}
```

add_one's activation record is destroyed.



General Tips

- Try to get all the easy points (Review the slides carefully)
- Try not to leave anything blank, so that we can find some points to give you
- There were quite a lot of exercised last year, so don't spend too much time on a particular one
- Do come to paper checking, for ECE280, usually you can find some points to be added.

References

[1] Zihao, He. VE280 Mid RC.

[3] Haoyang, Wu. VE280 Mid RC.

[4] Zhongqiang, Ren. Chapter 6-8.